

# DLR Dataspace

## Quick User Manual

### Asset & Contract Creation

#### Requirements:

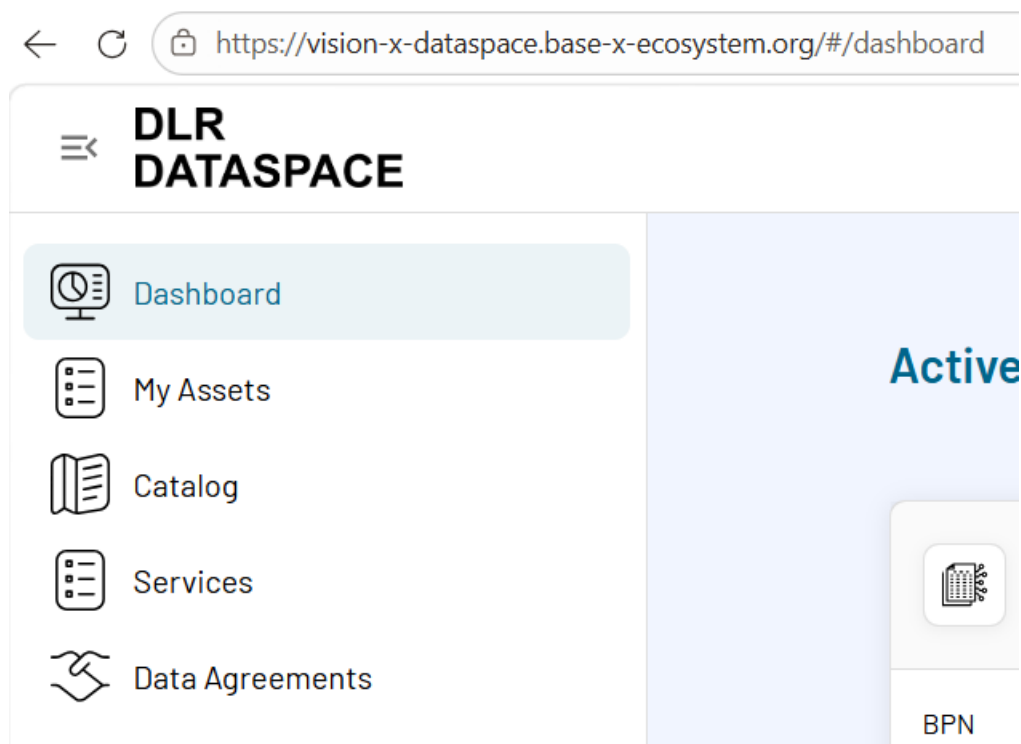
- DLR user account for login

#### Step 1. Login to the DLR dataspace

<https://vision-x-dataspace.base-x-ecosystem.org/#/home>

#### Step 1-1. Create a connector

If you are the first-time user, you need to create a connector. In the side-bar, click Dashboard. Then, create a HTTP type connector.



When the connector is successfully created and activated, you can see it as “running”



## Step 2. Get the API key and Connector Name

On the top right side of the screen, next to the profile icon, you can find 'API Key' button.



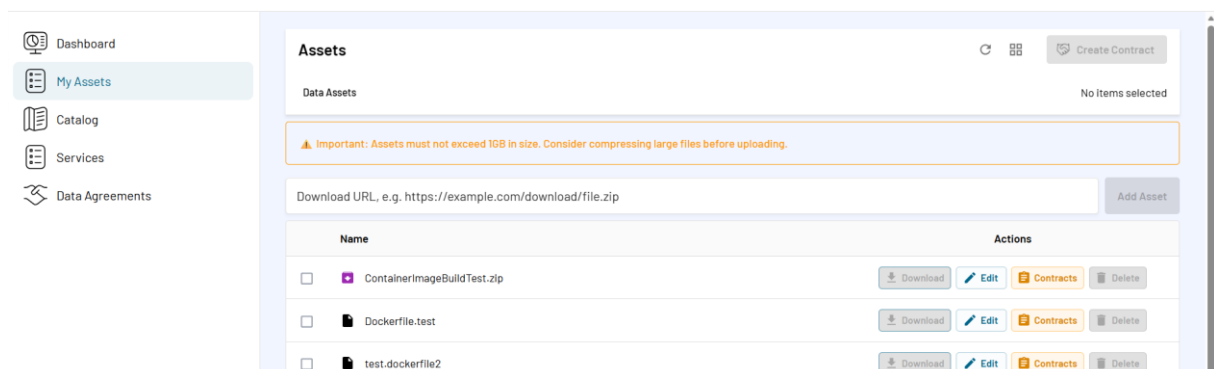
Also, you can find the name of your connector.

## Step 3. Create an HTTP asset

Navigate to 'My Assets' menu in the side-bar. Put an HTTP URL to make as an asset.

For this tutorial, we can use a publicly available URL:

<https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data>



Click the Add Asset button. In the pop-up window, give the file name 'Iris.csv'. Also, enable the checkbox *Include search engine metadata...* at the top.

Fill-in the metadata fields. You can use the “import JSON” button or the “Add Property” button.

Note that you can edit the metadata later.

Lastly, bind a policy to your asset. For now, select “Policy all”.

This means that anyone can see your asset and make a negotiation.

Create Offer

☒ Include search engine metadata (title, tags, description)

Source URL

https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data

Filename \*

Required. Use a file-style name like name.filetype.

Offer Metadata

Import JSON

Name

Optional. Overrides the stored asset name for this offer.

Title \*

Description \*

Domain

Tags

Custom Properties

Add property

Add additional property keys and values to the offer metadata. Reserved keys: name, filename, title, description, domain, tags.

No custom properties added yet.

Policy Selection

Policy

Policy all

Cancel

Create Offer

Once done, you will see the asset in your My Assets list.

The edit button on the right side allows you do edit the metadata of the asset.

The contracts button shows the contract information.

Note that you can make another contract with the same asset using the green button 'Create Contract' (e.g., with different policy)

The screenshot displays the 'Assets' management page. At the top, there's a 'Create Contract' button. Below it, a 'Data Assets' section shows '1 item selected'. A warning message states: 'Important: Assets must not exceed 1GB in size. Consider compressing large files before uploading.' Below this is a 'Download URL' input field and an 'Add Asset' button. The main table lists assets with columns 'Name' and 'Actions'. The asset 'iris.csv' is selected and highlighted with a red box. The 'Actions' column for 'iris.csv' shows buttons for 'Download', 'Edit', 'Contracts', and 'Delete'. The 'Contracts' button is highlighted in orange. Other assets listed include 'ContainerImageBuildTest.zip', 'Dockerfile.test', 'test.dockerfile2', 'test.dockerfile3', and 'vosk.dockerfile'.

| Name                                                 | Actions                                                                                                                                                    |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> ContainerImageBuildTest.zip | <input type="button" value="Download"/> <input type="button" value="Edit"/> <input type="button" value="Contracts"/> <input type="button" value="Delete"/> |
| <input type="checkbox"/> Dockerfile.test             | <input type="button" value="Download"/> <input type="button" value="Edit"/> <input type="button" value="Contracts"/> <input type="button" value="Delete"/> |
| <input checked="" type="checkbox"/> iris.csv         | <input type="button" value="Download"/> <input type="button" value="Edit"/> <input type="button" value="Contracts"/> <input type="button" value="Delete"/> |
| <input type="checkbox"/> test.dockerfile2            | <input type="button" value="Download"/> <input type="button" value="Edit"/> <input type="button" value="Contracts"/> <input type="button" value="Delete"/> |
| <input type="checkbox"/> test.dockerfile3            | <input type="button" value="Download"/> <input type="button" value="Edit"/> <input type="button" value="Contracts"/> <input type="button" value="Delete"/> |
| <input type="checkbox"/> vosk.dockerfile             | <input type="button" value="Download"/> <input type="button" value="Edit"/> <input type="button" value="Contracts"/> <input type="button" value="Delete"/> |

**\*Important: there is a bug:** As of 13 July 2026, it is found that the contracted asset is not visible in the frontend Catalog (in the side-bar menu). However, it is just a bug in the frontend, and the asset contract is actually listed in your catalog.

You can check this by using the dataspace API.

You can use the python script in the next page to check your own catalog.

Fill-in your connector name, BPN, and api-key before running the script.

Now, you can create more assets for your use case!

```
import requests
```

```
EDC_CONTEXT = {
```

```
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/2025/9/policy/",
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "odrl": "http://www.w3.org/ns/odrl/2/",
    "dcat": "http://www.w3.org/ns/dcat#",
    "dct": "http://purl.org/dc/terms/",
    "dspace": "https://w3id.org/dspace/v0.8/",
```

```
}
```

```
my_url = "https://vision-x-api.base-x-ecosystem.org/connectors/<<your-connector-name>>"
```

```
my_api_key = ""
```

```
my_bpn = ""
```

```
session = requests.Session()
```

```
session.headers.update({"Authorization": f"Bearer {my_api_key}"})
```

```
endpoint = "cp/management/v3/catalog/request"
```

```
url = f"{my_url}/{endpoint}"
```

```
payload = {
```

```
    "@context": EDC_CONTEXT,
    "counterPartyAddress": f'{str(my_url)}/cp/protocol',
    "counterPartyId": my_bpn,
    "protocol": "dataspace-protocol-http",
    "querySpec": {
        "offset": 0,
        "limit": 10000,
```

```
    },
```

```
}
```

```
response = session.post(url, json=payload)
```

```
response.raise_for_status()
```

```
print(response.json())
```